

QU Students Chat Bot

Automated Knowledge Base & AI Assistant

Supervisor: Dr. Abdulgader Al-Maimone

College/Dept: Qassim University – Computer Science

Term: 498

Team:

Suliman Saleh Al-Ghofaili (421109766)

Rayan Nayyaf Al-Harbi (421109529)

Khalifah Suliman Al-Khalifah (431109692)

Faris Ibrahim Al-Awaji (431109338)

Sultan Mohammed Al-Mutairi (421109632)

Abdulrahman Hesham Al-Resheed (421107788)

Key Features

24/7 Support

Technology

RAG Architecture

Languages

Arabic & English

Introduction: Need & Problem



Students struggle to get **fast, accurate answers** about admissions, registration, courses, and policies



Information is **scattered across multiple systems**; support channels are limited to working hours



Manual updates to knowledge sources lead to **outdated or inconsistent answers**

Goal

Deliver **24/7, reliable, verified responses** via an AI chatbot backed by an automated knowledge base

Research Questions

RQ1

How effective is a **RAG (retrieval-augmented)** chatbot in improving accuracy vs. keyword search?

RQ2

What pipeline best **automates knowledge updates** from official docs (e.g., Google Drive) to a vector DB?

RQ3

Which **workflow platform** (n8n, Activepieces, etc.) offers the best balance of flexibility, cost, and ease?

RQ4

What design choices improve **student experience** (latency, clarity, bilingual support)?

Literature Review (What we know)



University Chatbots

Improve access to information and **reduce administrative load**



RAG Architectures

Increase factuality by **grounding LLM answers** in retrieved documents



Workflow Automation

Helps keep data **fresh without heavy custom code**

Literature Review (Relevance to our RQs)



RAG Architecture RQ1/RQ4

RAG + **prompt discipline** ⇒ higher accuracy and lower hallucination



Knowledge Update Pipeline RQ2

Extract → chunk → embed → **upsert** enable continuous updates



Workflow Tools RQ3

Visual workflow tools (e.g., n8n) speed orchestration and reduce maintenance overhead

System Design (Architecture Overview)

Frontend

-  Web chat UI (Arabic/English)
-  Session memory
-  Citations/snippets

Backend API (Node.js)

-  /ask endpoint
-  Auth, rate-limit
-  Logging

RAG Pipeline



Ingest Docs



Parse & Chunk



Embed



Store



Retrieve

Orchestration

-  n8n for scheduled/triggered flows
-  Failure alerts

Deployment

-  Dockerized services
-  Environment secrets

System Design (Requirements)

✓ Functional Requirements

- FR1** Answer student questions **24/7** using latest approved content
- FR2** **Ingest new/updated files** from Google Drive automatically
- FR3** Provide **source citations** with each answer
- FR4** Support **Arabic & English** queries and responses
- FR5** Admin view: ingestion status, sync time, error logs
- FR6** Expose simple **REST /ask endpoint** for campus systems

📝 Non-Functional Requirements

- NFR1** Latency **≤ 3 seconds** p95 for common queries
- NFR2** Availability **≥ 99%** during term time
- NFR3** **Security**: API key management, role-based access
- NFR4** **Scalability**: handle **≥ 100** concurrent sessions
- NFR5** **Maintainability**: flows editable in n8n
- NFR6** **Observability**: metrics, timings, error alerts

Initial Screen Designs (Student Chat)

Home/Chat Screen

T

Header: "QU Students Chat Bot" + language toggle (AR/EN)



Input bar with example questions

? Example Questions

"Admission deadline?"
"Course registration rules"



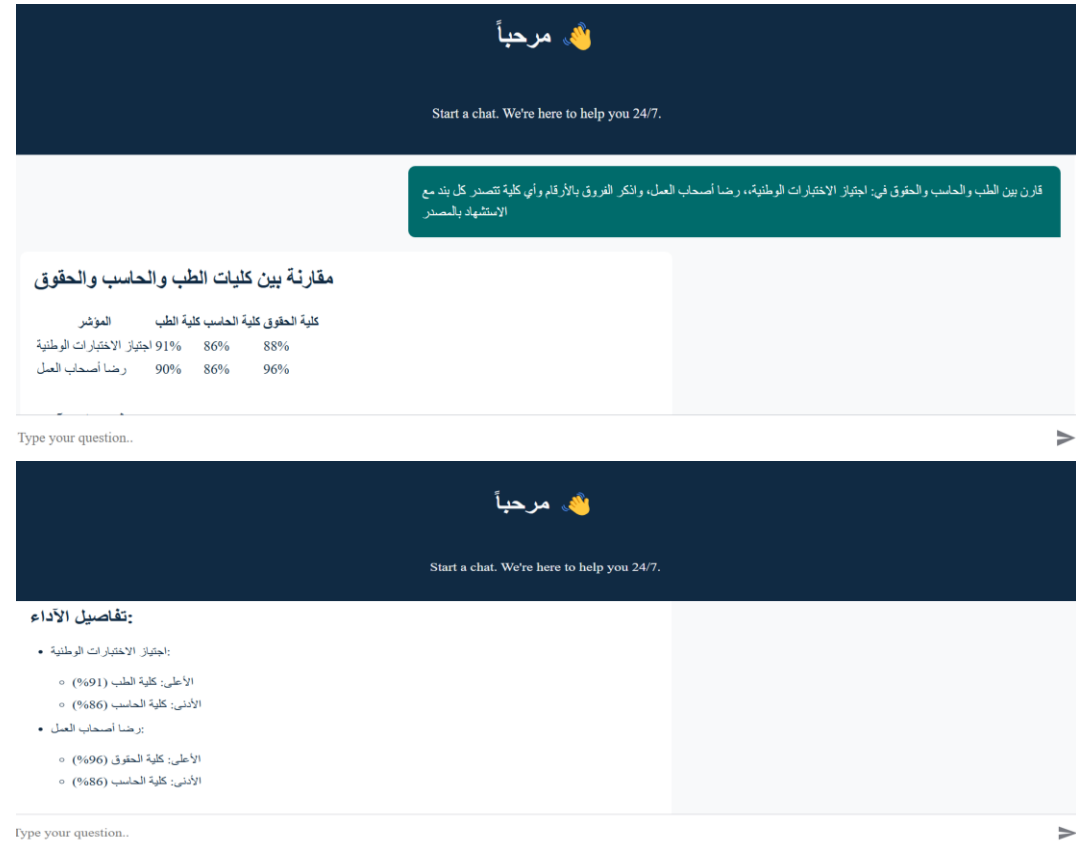
Message bubbles with **answer + cited sources**



"See details" expands snippet



Footer: Disclaimer "Answers sourced from official QU documents"



QU Students Chat Bot Interface Design (first version we will upgrade)

Initial Screen Designs (Admin & Flow)

Admin Dashboard

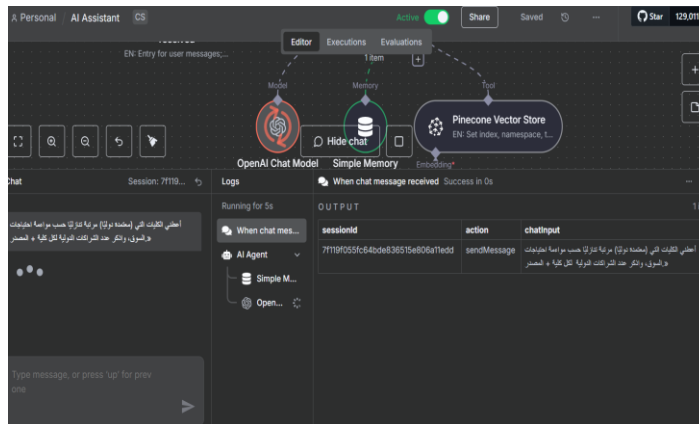
Dashboard Cards

- Last Sync
- Files Indexed
- Embedding Jobs
- Failures

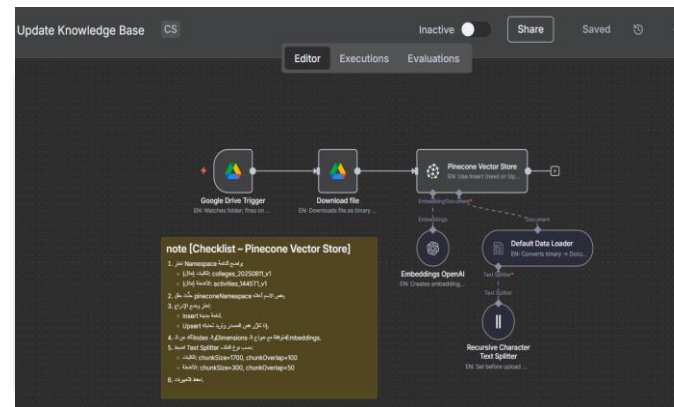
Action Buttons

- Re-sync
- Validate sources
- Export logs

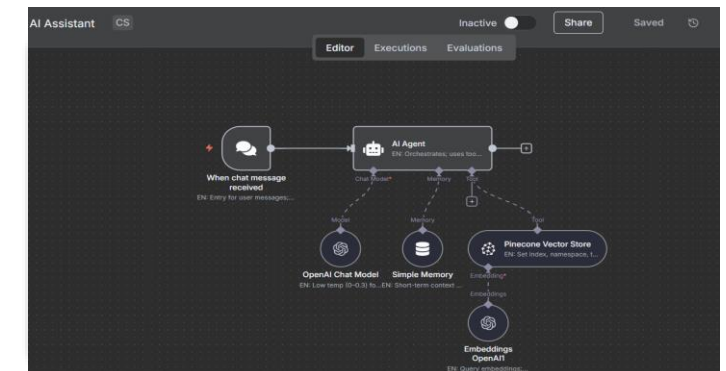
Admin Dashboard View



Workflow 1: Knowledge Base Update



Workflow 2: AI Agent & Chat Assist



Summary & Plan for 499

📄 What We Built/Learned



RAG improves accuracy and trust; automation keeps KB always fresh



Chosen stack: Node.js API, n8n orchestration, OpenAI embeddings, Pinecone vectors, Docker deploy

⚠️ Limitations

- Budgeted API costs
- Multilingual evaluation
- Campus SSO integration pending



Next (CS 499)

🔗 System Integration

- Connect with **University DB APIs** (SIS, timetable, course catalog)
- Implement **secure auth** and rate limits

📊 Data Expansion

- Expand data sources (SIS, course catalog, library)
- Unify schemas/metadata

📈 Analytics & UX

- Add advanced analytics & dashboards
- Perform user study (UX testing, p95 latency targets)